



```
In [145... import pandas as pd
```

```
In [146... df= pd.read_csv("insurance_data.csv")
```

```
In [147... df= df.sort_values(by='age')
```

```
In [148... df
```

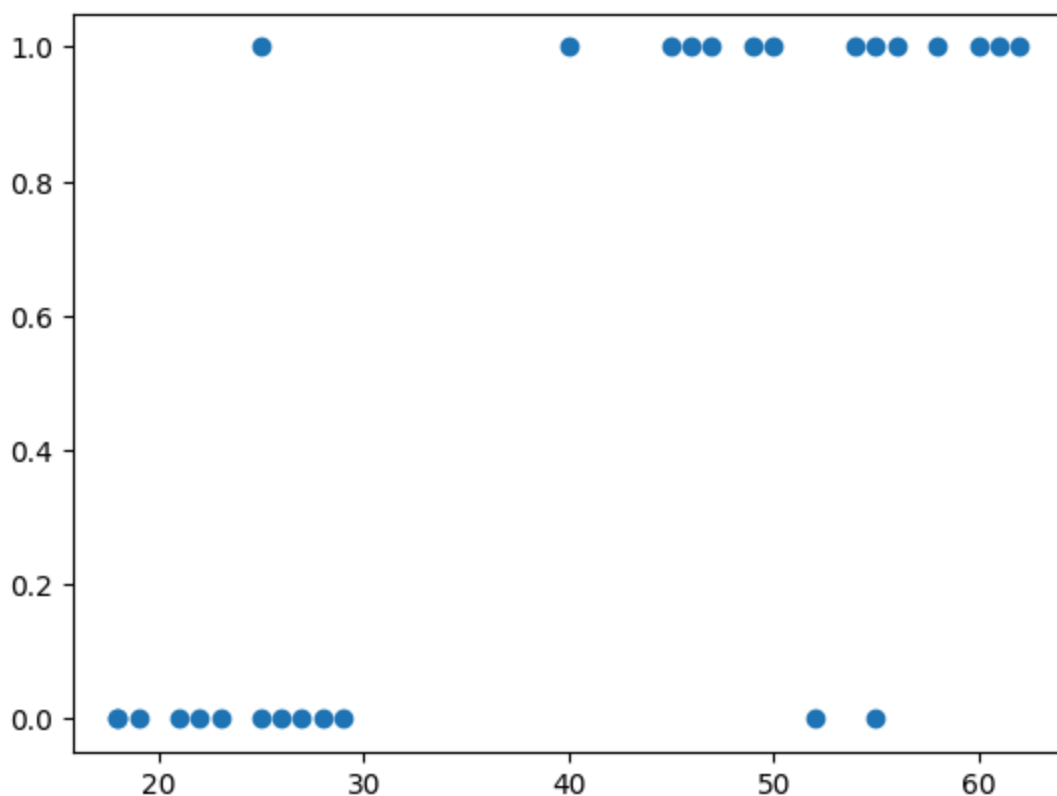
Out[148...

	age	bought_insurance
10	18	0
19	18	0
18	19	0
20	21	0
0	22	0
26	23	0
1	25	0
16	25	1
21	26	0
12	27	0
11	28	0
13	29	0
22	40	1
23	45	1
4	46	1
2	47	1
14	49	1
24	50	1
3	52	0
25	54	1
15	55	1
6	55	0
5	56	1
17	58	1
7	60	1
9	61	1
8	62	1

```
In [149... import matplotlib.pyplot as plt
```

```
In [150... plt.scatter(df.age,df['bought_insurance'] )
```

```
Out[150... <matplotlib.collections.PathCollection at 0x186617e34d0>
```



```
In [151... from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
In [152... model= LinearRegression()
```

```
In [153... x= df[["age"]]
y= df['bought_insurance']
```

```
In [154... x_train, x_test, y_train , y_test= train_test_split(x, y, test_size=0.2, random_state=42)
```

```
In [155... model.fit(x_train, y_train)
```

```
Out[155... ▼ LinearRegression ⓘ ?
```

Parameters

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

```
In [156... model.score(x_test, y_test)
```

```
# model.score(x_train, y_train)
```

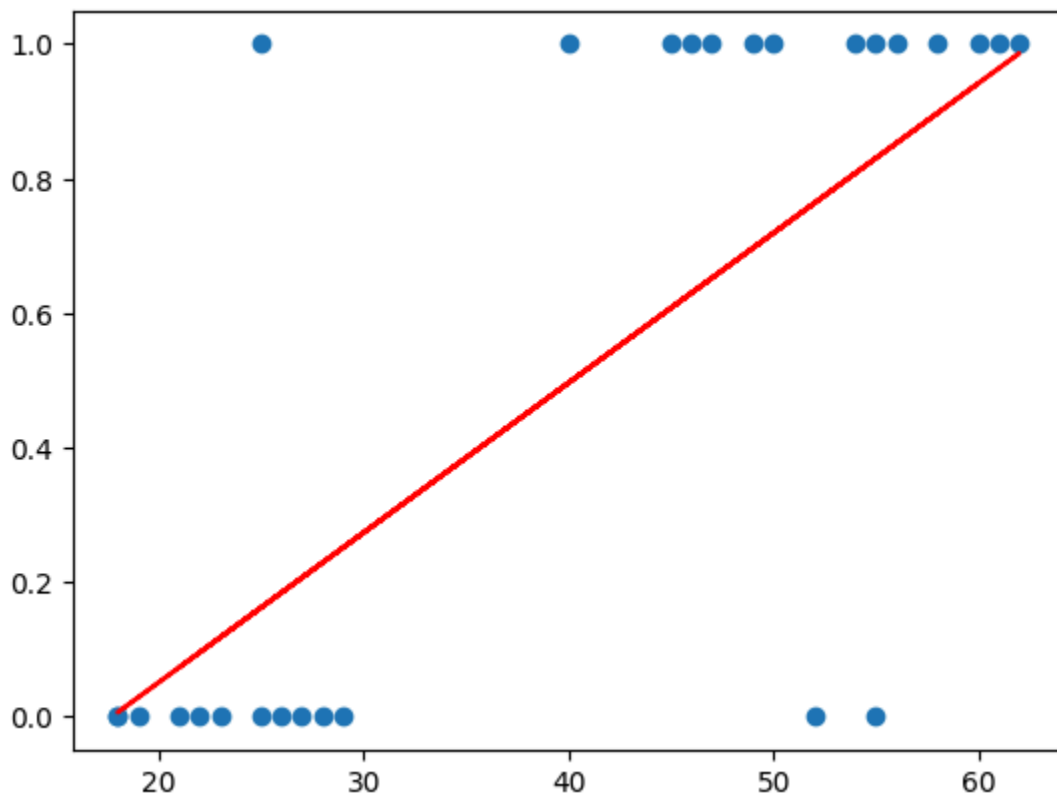
Out[156...] 0.6860904332432788

```
In [157...] y_pred= model.predict(x_train)
y_pred
```

Out[157...] array([0.16231972, 0.07324829, 0.71901616, 0.94169474, 0.62994473,
0.09551614, 0.22912329, 0.117784 , 0.8080876 , 0.00644471,
0.02871257, 0.16231972, 0.89715903, 0.83035545, 0.76355188,
0.25139115, 0.85262331, 0.98623046, 0.60767687, 0.65221259,
0.18458758])

```
In [158...] plt.scatter(x['age'], y)
plt.plot(x_train['age'], y_pred,color="red")
```

Out[158...] [<matplotlib.lines.Line2D at 0x186618202d0>]



```
In [159...] from sklearn.linear_model import LogisticRegression
```

```
In [160...] model2= LogisticRegression()
```

```
In [161...] model2.fit(x_train, y_train)
```

Out[161...

▼ LogisticRegression ⓘ ?	
Parameters	
penalty	'deprecated'
C	1.0
l1_ratio	0.0
dual	False
tol	0.0001
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	100
verbose	0
warm_start	False
n_jobs	None

In [162... `model2.score(x_test, y_test)`

Out[162... 0.8333333333333334

In [163... `x_test`

Out[163...

	age
9	61
10	18
12	27
15	55
14	49
22	40

In [164... `model2.score(x_train, y_train)`

Out[164... 0.8571428571428571

```
In [165... model2.predict_proba(x_test)
```

```
Out[165... array([[0.08144876, 0.91855124],  
          [0.93673843, 0.06326157],  
          [0.83533604, 0.16466396],  
          [0.15333356, 0.84666644],  
          [0.27001261, 0.72998739],  
          [0.51915068, 0.48084932]])
```

```
In [166... x_test
```

```
Out[166... 

|           | age |
|-----------|-----|
| <b>9</b>  | 61  |
| <b>10</b> | 18  |
| <b>12</b> | 27  |
| <b>15</b> | 55  |
| <b>14</b> | 49  |
| <b>22</b> | 40  |


```

```
In [167... x
```

Out[167...

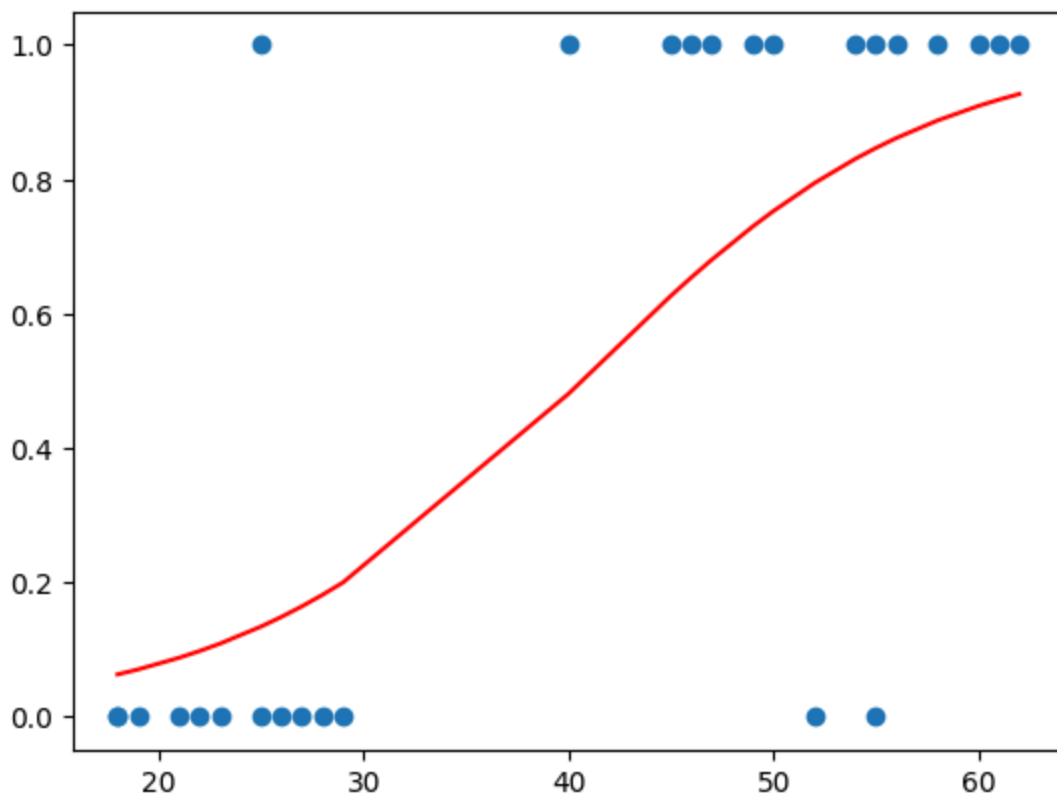
	age
10	18
19	18
18	19
20	21
0	22
26	23
1	25
16	25
21	26
12	27
11	28
13	29
22	40
23	45
4	46
2	47
14	49
24	50
3	52
25	54
15	55
6	55
5	56
17	58
7	60
9	61
8	62

```
In [168... y_pred2= model2.predict_proba(x)
y_pred2
```

```
Out[168... array([[0.93673843, 0.06326157],
                  [0.93673843, 0.06326157],
                  [0.92930775, 0.07069225],
                  [0.91198047, 0.08801953],
                  [0.90194613, 0.09805387],
                  [0.8909047 , 0.1090953 ],
                  [0.86552658, 0.13447342],
                  [0.86552658, 0.13447342],
                  [0.85106131, 0.14893869],
                  [0.83533604, 0.16466396],
                  [0.81830494, 0.18169506],
                  [0.7999342 , 0.2000658 ],
                  [0.51915068, 0.48084932],
                  [0.37321001, 0.62678999],
                  [0.34581362, 0.65418638],
                  [0.3194035 , 0.6805965 ],
                  [0.27001261, 0.72998739],
                  [0.24720394, 0.75279606],
                  [0.20560486, 0.79439514],
                  [0.16943047, 0.83056953],
                  [0.15333356, 0.84666644],
                  [0.15333356, 0.84666644],
                  [0.13851092, 0.86148908],
                  [0.11246984, 0.88753016],
                  [0.09080861, 0.90919139],
                  [0.08144876, 0.91855124],
                  [0.07297621, 0.92702379]])
```

```
In [169... plt.scatter(x['age'], y)
plt.plot(x['age'],y_pred2[:,1] , color= "red")
```

```
Out[169... [<matplotlib.lines.Line2D at 0x186617efb10>]
```



```
In [170... from sklearn.datasets import load_digits
          digits= load_digits()
```

```
In [171... dir(digits)
```

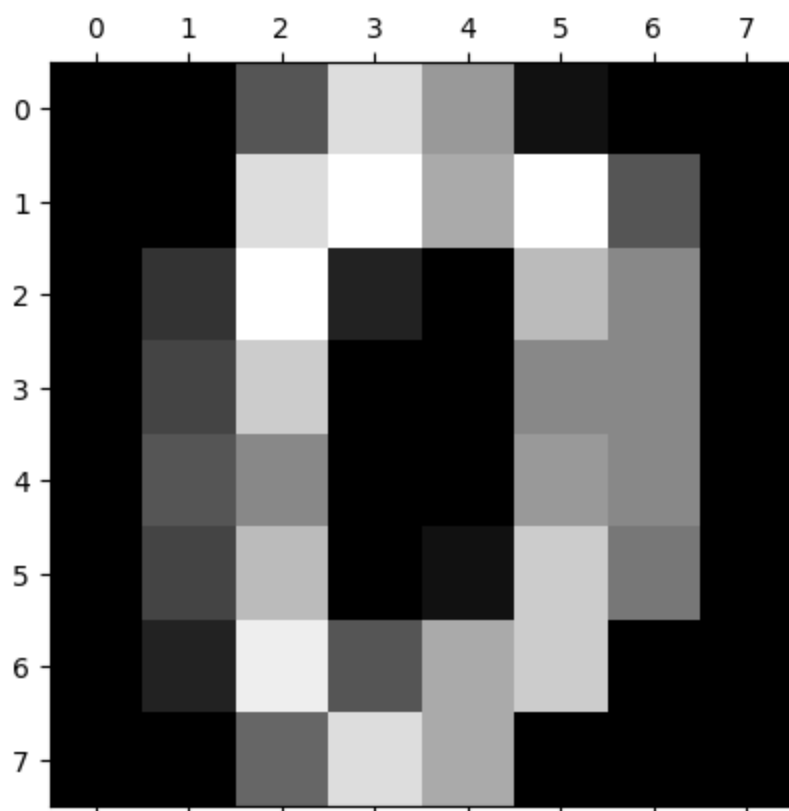
```
Out[171... ['DESCR', 'data', 'feature_names', 'frame', 'images', 'target', 'target_name
s']
```

```
In [172... digits.data
```

```
Out[172... array([[ 0.,  0.,  5., ...,  0.,  0.,  0.],
                [ 0.,  0.,  0., ..., 10.,  0.,  0.],
                [ 0.,  0.,  0., ..., 16.,  9.,  0.],
                ...,
                [ 0.,  0.,  1., ...,  6.,  0.,  0.],
                [ 0.,  0.,  2., ..., 12.,  0.,  0.],
                [ 0.,  0., 10., ..., 12.,  1.,  0.]], shape=(1797, 64))
```

```
In [173... plt.gray()
          plt.matshow(digits.images[0])
```

```
Out[173... <matplotlib.image.AxesImage at 0x186618b59d0>
          <Figure size 640x480 with 0 Axes>
```



```
In [174... x= digits.data  
          y= digits.target
```

```
In [175... y
```

```
Out[175... array([0, 1, 2, ..., 8, 9, 8], shape=(1797,))
```

```
In [176... digits.target_names
```

```
Out[176... array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [177... x_train,x_test, y_train , y_test= train_test_split(x,y, test_size=0.2, random_
```

```
In [178... x_test.shape
```

```
Out[178... (360, 64)
```

```
In [179... x_train.shape
```

```
Out[179... (1437, 64)
```

```
In [180... model3= LogisticRegression()
```

```
In [181... model3.fit(x_train, y_train)
```

```
c:\Users\rohan\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\linear_model\_logistic.py:406: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
on
n_iter_i = _check_optimize_result(

Out[181]...

▼ LogisticRegression ⓘ ?		
Parameters		
	penalty	'deprecated'
	C	1.0
	l1_ratio	0.0
	dual	False
	tol	0.0001
	fit_intercept	True
	intercept_scaling	1
	class_weight	None
	random_state	None
	solver	'lbfgs'
	max_iter	100
	verbose	0
	warm_start	False
	n_jobs	None

```
In [182]... model3.score(x_test, y_test)
```

Out[182]... 0.9444444444444444

```
In [183]... model3.score(x_train, y_train)
```

Out[183]... 1.0

```
In [184]... model3.predict([x_test[4]])
```

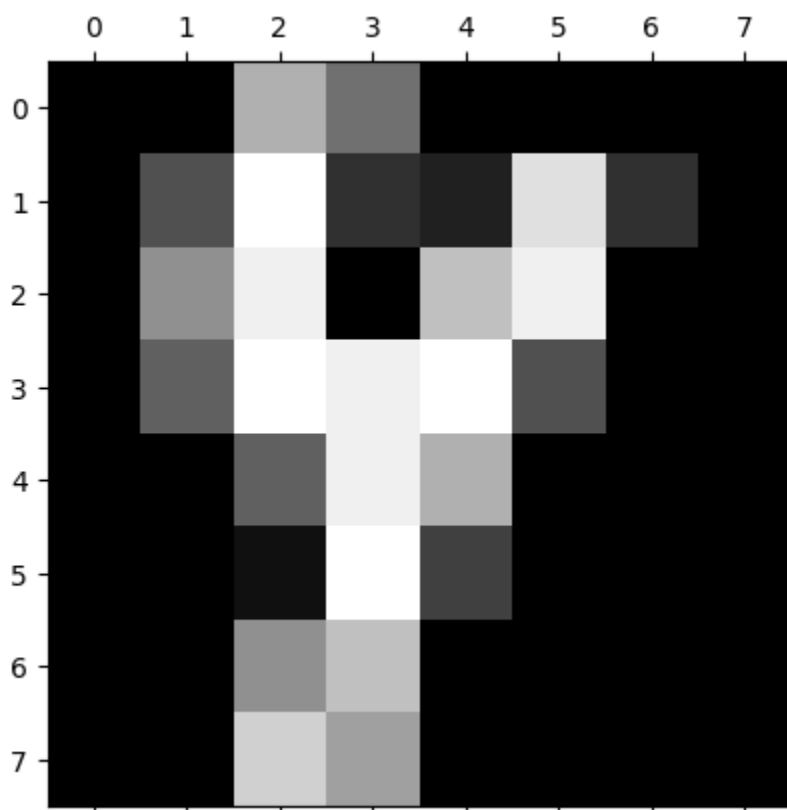
```
Out[184...] array([8])
```

```
In [185...] x_test[0].shape
```

```
Out[185...] (64,)
```

```
In [194...] plt.matshow(x_test[4].reshape(8,8))
```

```
Out[194...] <matplotlib.image.AxesImage at 0x18661daa290>
```



```
In [187...] model3.predict_proba([x_test[0]])
```

```
Out[187...] array([[6.18240610e-16, 3.55422973e-05, 5.60403448e-21, 9.14392483e-18,  
                    9.99964380e-01, 2.01712308e-18, 6.96794550e-11, 7.69362096e-08,  
                    2.89311337e-10, 4.19155044e-19]])
```

```
In [188...] from sklearn.metrics import confusion_matrix
```

```
In [189...] y_pred= model3.predict(x_test)
```

```
In [190...] cm= confusion_matrix(y_test,y_pred)
```

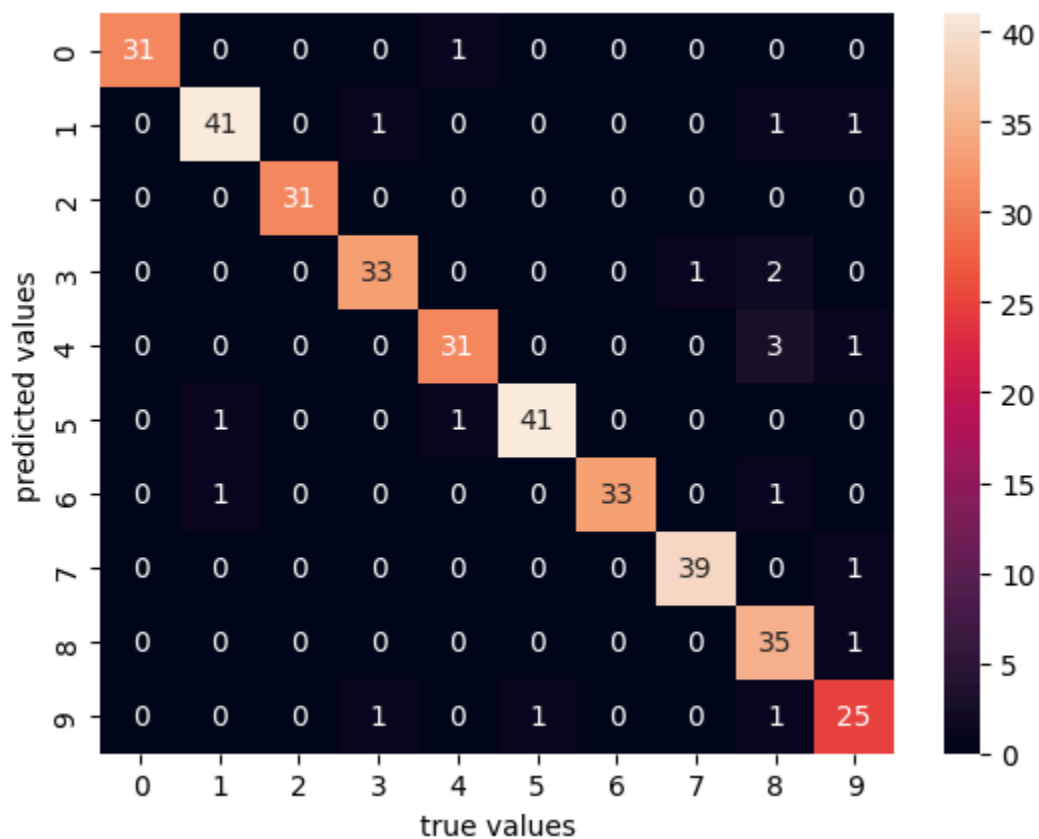
```
In [191...] cm
```

```
Out[191...] array([[31,  0,  0,  0,  1,  0,  0,  0,  0,  0],
 [ 0, 41,  0,  1,  0,  0,  0,  0,  1,  1],
 [ 0,  0, 31,  0,  0,  0,  0,  0,  0,  0],
 [ 0,  0,  0, 33,  0,  0,  0,  1,  2,  0],
 [ 0,  0,  0,  0, 31,  0,  0,  0,  3,  1],
 [ 0,  1,  0,  0,  1, 41,  0,  0,  0,  0],
 [ 0,  1,  0,  0,  0,  0, 33,  0,  1,  0],
 [ 0,  0,  0,  0,  0,  0,  0, 39,  0,  1],
 [ 0,  0,  0,  0,  0,  0,  0,  0, 35,  1],
 [ 0,  0,  0,  1,  0,  1,  0,  0,  1, 25]])
```

```
In [192...] import seaborn as sns
```

```
In [193...] sns.heatmap(cm, annot=True)
plt.xlabel("true values")
plt.ylabel("predicted values")
```

```
Out[193...] Text(50.72222222222214, 0.5, 'predicted values')
```



```
In [ ]:
```